



On recent computational results for a dynamic pickup and delivery problem

Markó Horváth¹ · Tamás Kis¹

Received: 26 April 2024 / Accepted: 15 April 2025 / Published online: 26 May 2025
© The Author(s) 2025

Abstract

A few years ago, a dynamic pickup-and-delivery problem was introduced in the context of a competition Hao et al. (ICAPS 2021 Competition). Since then, the problem has attracted the attention of many researchers. Recently, Zhou et al. (Memet Computing 16:1-8, 2014) proposed a solution approach to the problem, however, we found a flaw in their study that leads to a lot of misunderstanding. Our paper aims to clear up these misunderstandings. In this paper, we state and prove that contrary to their claim, Zhou et al. (Memet Computing 16:1-8, 2014) did not study and solve the original problem, but a relaxation of it. Accordingly, but without mentioning it, the authors modified the benchmark dataset and then applied state-of-the-art methods tailored to the original problem. Therefore, their published results are misleading as the authors did not clarify that the results are not for the original problem but for a relaxation.

Keywords Dynamic pickup and delivery problem · ICAPS 2021 competition

1 Introduction

A dynamic pickup-and-delivery problem was introduced in a competition organized by the International Conference on Automated Planning and Scheduling in 2021 (ICAPS 2021), see [5]. We will refer to this problem as ICAPS-DPDP. Since then, several papers dealt with this problem (e.g., Cai et al. [1–3]; Du et al. [4]). Recently, Zhou et al. [6] proposed a memetic algorithm for the problem, and compared their approach to some algorithms originally developed for the ICAPS-DPDP on the widely used benchmark dataset. In particular, they compared their results to that obtained by the gold, silver and bronze teams' algorithm of the ICAPS 2021 competition. We observed that the scores provided in that paper are sometimes an order of magnitude better than those obtained by other researchers. We tried to understand the reason of the huge differences, and this paper reports on our findings.

In this paper, we will demonstrate that Zhou et al. [6] ignored a very important constraint of the problem. Accord-

ingly, the authors used not the original, but a modified benchmark dataset for their computational experiments. Since the authors did not emphasize that they were studying a relaxed problem, or that they were testing methods for the original problem on a modified dataset, their published results are misleading. Our claims are the following.

1. Contrary to their claim, Zhou et al. [6] did not study and solve the ICAPS-DPDP, but a relaxation of it, where the number of docking ports at the factories is unlimited.
2. The constraints that Zhou et al. [6] ignored have a huge impact on the feasibility of the solutions and on the objective function value.
3. For their computational experiments, Zhou et al. [6] modified the benchmark tools to make them applicable to their relaxed problem and their proposed method. Since this is not stated in any way in their publication, their provided computational results are misleading to those who would investigate the original problem. The existing methods for the original problem were also tested on this modified dataset. Therefore the results provided by the authors are not consistent with the ones published by other researchers.

✉ Markó Horváth
marko.horvath@sztaki.hu

Tamás Kis
tamas.kis@sztaki.hu

¹ HUN-REN Institute for Computer Science and Control,
Kende u. 13-17, Budapest H-1111, Hungary

To prove our claims, we performed computational experiments with some publicly available algorithms on the

benchmark dataset. The results were not consistent with those published in Zhou et al. [6]. Then, we modified the benchmark dataset according to our suspicions, and repeated the experiments. The results confirmed our claims.

Structure of the paper In Sect. 2, we briefly present the original problem, and in Sect. 3 we explain in detail how the problem was modified by Zhou et al. [6]. We provide numerical evidence proving our claims in Sect. 4.

2 The ICAPS-DPDP

Now, we briefly present the ICAPS-DPDP (Sect. 2.1) introduced in the ICAPS 2021 competition [5], and the related benchmark tools (Sect. 2.2).

2.1 Problem overview

There is a fleet of homogeneous vehicles that has to serve pickup-and-delivery order requests which occur over a day. Each order is characterized by a quantity, a pickup factory, a delivery factory, a release time, and a due date. The vehicles can be loaded up to their capacity, while unloading has to follow the last-in-first-out (LIFO) rule. Those, but only those orders whose quantity exceeds the capacity of the vehicles, can be split and delivered separately. The travel times and the distances between the factories are given. Each factory has a given number of docking ports for serving (that is, loading and unloading) the vehicles. Vehicles are served on a first-come-first-served basis. If a vehicle arrives at a factory and all ports are occupied, its service cannot begin immediately, but the vehicle has to join the waiting queue. That is, the vehicle must wait until one of the docking ports becomes free, and no vehicle that arrived earlier is waiting for a port. The objective is to satisfy all the requests such that a combination of tardiness penalties and traveling distances is minimized. Note that the waiting at the factories has a huge impact on the solution quality, as it may cause tardiness.

The problem is *dynamic* because the order requests arrive real-time, i.e., the orders are unknown until their release time. Accordingly, the routes of the vehicles have to be determined and updated dynamically. An algorithm for the dynamic problem is provided with the state of the system, and computes the routes of the vehicles. A state describes the information about the orders already released, and about the vehicles such as their current locations and the orders they are currently carrying.

2.2 Benchmark tools

The organizers of the competition provided the participants with a public benchmark dataset and a simulator to enable

Table 1 Dataset settings

Instance set	#Instances	#Vehicles	#Orders	#Ports
Set 1	8	5	50	6
Set 2	8	5	100	6
Set 3	8	20	300	6
Set 4	8	20	500	6
Set 5	8	50	1000	6
Set 6	8	50	2000	6
Set 7	8	100	3000	6
Set 8	8	100	4000	6

dynamic testing of their routing algorithms.¹ The algorithms implemented by the top three teams are also publicly available.²

Benchmark dataset The benchmark dataset consists of a factory info file, a route info file, and 64 problem instances. The factory info file describes the 153 factories, including the number of their available ports, while travel times and distances between the factories are given in the route info file. The problem instance files contain the data for the vehicles and for the orders. The instances can be divided into 8 sets based on their basic characteristics, see Table 1.

Benchmark simulator The benchmark simulator is initialized with the factory and route info files, and with the given problem instance, after which the simulation starts. The simulator invokes the routing algorithm at every 10 min (in simulation time) with the current state. In response, the algorithm returns the updated vehicle routes. This process is repeated until all orders are delivered, after which the simulator provides the *score* of the solution, which is the weighted sum of the total distance traveled by the vehicles and the tardiness of the orders.

Existing algorithms

The algorithms of the top three teams are considered as state-of-the-art solution approaches for the ICAPS-DPDP. We will not go into the details of these methods, except to point out a fact that will be important later. Briefly stated, due to the 10 min update frequency of the simulator, a shorter time limit may have been set in the algorithms. The silver and bronze prize algorithms have relatively short execution times, so the time limit, if set at all, did not affect the final solutions. Therefore, these methods can be considered *deterministic*, i.e., they produce the same solutions when run in different environments. This is supported by the fact that in the former publications on the ICAPS-DPDP, all authors obtained the same results with these algorithms (cf. [1, Tables 4-6], [2,

¹ https://github.com/huawei-noah/xingtian/tree/master/simulator/dpdp_competition

² <https://competition.huaweicloud.com/information/1000041411/Winning>

Table 2], [3, Tables 2–4]). (The exception is [6, Table 8], which motivated the writing of the paper at hand). In contrast, the local search approach of the gold prize team often reaches their 9-minute time limit per decision epoch, thus different runs may give different results. Therefore, this method is considered *non-deterministic*.

3 The problem variant of Zhou et al. [6]

Zhou et al. [6] claimed that they focus on the problem that was presented in the competition, i.e., the ICAPS-DPDP (referred to as *real-world DPDP* in their paper). The authors also emphasized this in the computational part: "Since we focus on the same DPDP problem as these three algorithms, we regard them as state-of-the-art algorithms and utilize them as baseline algorithms".

Despite the above, the authors omitted one of the key features of the ICAPS-DPDP in their problem definition, namely, that factories have a limited number of docking ports, which may force vehicles to wait (Sect. 2.1). The authors merely said that "each factory has several cargo docks", and there is no mention at all that the number of docking ports is limited, and that the vehicles may have to wait. Their formula for the time that a vehicle spends at a factory [6, Constraint (6)] also confirms the lack of these docking capacity constraints. Thus, we claim that Zhou et al. [6] studied not the ICAPS-DPDP, but only a relaxation of the problem. In Sect. 4 we will prove our claims.

A further, much less significant issue is that when calculating order delay, Zhou et al. [6, Constraint (9)] consider the end of the unloading of the corresponding order. In contrast, in case of the ICAPS-DPDP, the orders are already considered delivered when the vehicle arrives at the delivery factory. However, it has no consequence for the score of a solution, since it is calculated by the simulator independently of the routing algorithm.

4 Computational results

The aim of our computational experiments was to prove our claims, i.e., to show that Zhou et al. [6] solved a relaxation of the ICAPS-DPDP, and the authors also modified the benchmark tools for their experiments. For that, we used the same publicly available benchmark dataset and implemented algorithms as the authors. The idea was simply to run these methods on the original dataset and on the dataset we modified according to our suspicions, then compare the results with those published in [6].

Setup of the experiments

We used the same 64 benchmark instances and the same simulator that was provided to the participants of the compe-

tion, as presented in Sect. 2.2. For the routing, we used the algorithms of the silver and the bronze prize team. All the data and codes we used for these experiments are available online, so our tests can be easily reproduced.

Methodology

First, we ran the algorithms of the silver and bronze prize team on the original instances, where each factory has a limited number of docking ports, namely, 6 for each factory. Then, we modified the factory info file to increase the number of docking ports uniformly to 100, and repeated the experiments. Since the number of available vehicles never exceeds 100 (see Table 1), this modification models the case where the number of docking ports is unlimited.

Results

Table 2 shows how the silver and bronze prize methods performed on the *original* instances and on the *modified* instances. The scores reported in [6, Table 8] on the same dataset with the same algorithms are also indicated in the table. Our detailed results can be found in Table 3.

Observation 1 The summarized results provided by [6] are completely the same as our results on the modified instances (i.e., the unlimited case). This proves that, in contrast to the original problem definition, [6] considered an unlimited number of docking ports at the factories. Therefore, the authors studied not the ICAPS-DPDP, but only a relaxation of it. Moreover, the authors also modified the benchmark tools (either the instances like us or the simulator itself), when testing the existing methods tailored to the original problem.

Observation 2 It is clear that the number of docking ports has a huge impact on the objective value. In the case of instance sets 1–2, the number of vehicles is already less than the number of docking ports available at the factories (Table 1), so the modification has no effect. However, as the number of vehicles and orders increases, the vehicles are forced to wait at the factories, causing a lot of waiting time. Assuming an unlimited number of docking ports, we achieved 95 % and 88 % better results with the silver and bronze algorithms, respectively. We can therefore state that the original problem and its relaxation are significantly different. Consequently, the state-of-the-art routing algorithms for the ICAPS-DPDP cannot be considered as state-of-the-art algorithms for the modified problem of [6].

Remark 1 We used only two of the four methods for which Zhou et al. [6] provided computational results. First, we did not use the algorithm of the gold prize team because it is not deterministic (see Sect. 2.2), therefore it is not possible to reproduce the exactly same average results as provided in [6, Table 8]. Second, we did not use the memetic algorithm proposed by Zhou et al. [6] because it is not publicly available, and it is also non-deterministic. Nevertheless, the other two methods were enough to confirm our suspicions.

Table 2 Summarized results obtained with the silver and bronze prize algorithms. The values reported by Zhou et al. [6, Table 8] for these two algorithms are also indicated

Instances	Silver (IM) ²			Bronze (LS) ³		
	Our experiments		Zhou et al. [6]	Our experiments		Zhou et al. [6]
	Original	Modified ¹		Original	Modified ¹	
Set 1	13 676.25	13 676.25	13 676.25	1 763.83	1 763.83	1 763.83
Set 2	599 932.78	599 932.68	599 932.78	62 180.16	62 180.17	62 180.17
Set 3	2 310.72	2 310.71	2 310.72	8 969.69	10 110.14	10 110.14
Set 4	105 049.36	105 049.35	105 049.36	26 938.30	28 297.94	28 297.94
Set 5	17 284.30	17 683.53	17 683.52	94 794.85	73 412.50	73 412.50
Set 6	153 419.12	137 412.22	137 412.22	651 944.95	349 933.19	349 933.19
Set 7	904 586.31	40 785.66	40 785.67	1 941 385.33	596 060.01	596 060.01
Set 8	18 678 529.15	75 814.69	75 814.67	15 122 816.25	1 091 696.33	1 091 696.33
Avg	2 559 348.50	124 083.14	124 083.15	2 238 849.17	276 681.76	276 681.76

¹modified instances – unlimited number of docking ports²silver prize algorithm – abbreviated as IM in [6]³bronze prize algorithm – abbreviated as LS in [6]**Table 3** Detailed results obtained with the silver and bronze prize algorithms on the original and on the modified instances

Instance	Silver (IM)		Bronze (LS)	
	Original	Modified	Original	Modified
1	2 303.59	2 303.59	130.20	130.20
2	30 478.90	30 478.90	91.40	91.42
3	35 800.72	35 800.72	96.50	96.54
4	5 491.58	5 491.58	104.20	104.18
5	16 928.79	16 928.79	5 446.30	5 446.28
6	4 761.87	4 761.87	118.10	118.08
7	12 847.39	12 847.39	7 355.40	7 355.39
8	797.11	797.11	768.50	768.52
9	181 179.60*	181 179.60*	8 481.40	8 481.44
10	1 766 275.80*	1 766 275.00*	187 123.30	187 123.26
11	180 883.50	180 883.50	3 041.60	3 041.61
12	567 136.40*	567 136.40*	84 158.30	84 158.32
13	220 249.20*	220 249.20*	277.30	277.27
14	236 920.60*	236 920.60*	7 824.40	7 824.43
15	792 942.10*	792 942.10*	148 730.60	148 730.64
16	853 875.00*	853 875.00*	57 804.40	57 804.37
17	96.48	96.50	411.10	411.14
18	111.54	111.50	8 557.80	10 254.39
19	477.69	477.70	4 150.20	4 150.20
20	3 984.23	3 984.20	16 224.40	16 224.41
21	2 341.34	2 341.30	18 501.40	18 501.39
22	3 320.96	3 321.00	21 102.60	21 102.62
23	6 865.84	6 865.80	808.90	808.88
24	1 287.69	1 287.70	2 001.10	9 428.11
25	92 380.91	92 380.90	21 451.30	24 070.75
26	275 468.88	275 468.90	53 998.50	53 998.47
27	18 439.83	18 439.80	18 812.00	17 760.98
28	9 444.74	9 444.70	14 868.00	14 867.96
29	162 951.92	162 951.90	42 397.80	40 338.77

Table 3 continued

Instance	Silver (IM)		Bronze (LS)	
	Original	Modified	Original	Modified
30	74026.88	74026.90	21113.30	20279.66
31	147256.33	147256.30	23311.90	32594.38
32	60425.37	60425.40	19553.60	22472.58
33	7031.30	7031.30	95017.10	57708.90
34	10503.40	10503.40	54373.60	50365.80
35	15760.60	15760.60	103753.30	44314.25
36	23186.20	23186.20	112878.60	111794.31
37	5141.80	5141.80	95739.60	37494.98
38	30136.40	30136.30	94229.90	86580.14
39	22253.00	22252.40	85226.10	92519.27
40	24261.70	27456.20	117140.60	106522.37
41	124253.30	93265.10	548755.60	422615.02
42	164002.20	145392.40	600981.80	233253.26
43	166646.40	170619.30	550660.70	445425.43
44	190604.40	124446.90	715480.90	509249.51
45	181170.60	166700.40	616244.70	332937.88
46	176490.70	218913.80	798330.90	280641.27
47	104337.20	95512.60	602968.10	334238.94
48	119848.20	84447.30	782136.90	241104.19
49	1184048.30	65398.40	1594689.90	619541.97
50	572493.20	28963.60	2512247.10	702290.81
51	515466.90	47035.00	1335194.40	613350.81
52	447647.30	36449.30	2030603.40	604531.98
53	316974.00	41996.60	1930298.90	556281.89
54	1629783.60	47312.50	1995016.00	622635.39
55	729537.30	34597.60	1975595.90	478129.65
56	1840739.90	24532.30	2157437.00	571717.56
57	22433901.40	66942.40	16517326.30	996135.28
58	13243401.10	71211.00	11214248.90	1250928.24
59	12351487.90	83130.90	9542269.40	1215468.30
60	21521731.90	114287.90	13191277.80	1355281.44
61	9095705.50	72155.90	7207770.30	896033.22
62	14541495.90	82006.40	10857350.80	1053966.23
63	30883610.50	61902.40	25025232.80	950872.83
64	25356899.00	54880.60	27427053.70	1014885.07

*results obtained with a minimal modification of the simulator

Remark 2 The silver prize algorithm failed on 7 out of 8 instances of Set 2 due to a technical constraint of the simulator. Thus, in case of that algorithm and those instances, we modified the simulator by removing lines 128-130 of the file `simulate_environment.py`.

5 Conclusions

In this paper, we criticized a recent paper on a practical dynamic pickup and delivery problem, called the ICAPS-DPDP. We proved with easily reproducible computational experiments that, contrary to their claim, Zhou et al. [6] dealt with not the ICAPS-DPDP, but only a relaxation of the problem. Our results show that the original version of the problem and that variant significantly differ from each other. We also showed that Zhou et al. [6] modified the benchmark

tools when testing existing algorithms tailored to the original problem. Therefore, their provided computational results are misleading to those who would investigate the original problem.

Note, however, that the methodology the authors used for the relaxed problem is reasonable. The mistake was made in not emphasizing (not mentioning at all) that they were not looking at the original problem, but at the relaxed problem, and thus publishing misleading results.

Acknowledgements This work was supported by the National Research, Development and Innovation Office, grant numbers TKP2021-NKTA-01. Markó Horváth acknowledges the support of the János Bolyai Research Scholarship.

Funding Open access funding provided by HUN-REN Institute for Computer Science and Control.

Data Availability The data that support the findings of this study are openly available at the websites linked in the article.

Declarations

Conflict of interest The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by/4.0/>.

References

1. Cai J, Zhu Q, Lin Q (2022) Variable neighborhood search for a new practical dynamic pickup and delivery problem. *Swarm Evol Comput* 75:101182
2. Cai J, Zhu Q, Lin Q, Li J, Chen J, Ming Z (2022) An efficient multi-objective evolutionary algorithm for a practical dynamic pickup and delivery problem. In: *international conference on intelligent computing*, pp. 27–40. Springer
3. Cai J, Zhu Q, Lin Q, Ma L, Li J, Ming Z (2023) A survey of dynamic pickup and delivery problems. *Neurocomputing* 554:126631
4. Du J, Zhang Z, Wang X, Lau HC (2023) A hierarchical optimization approach for dynamic pickup and delivery problem with LIFO constraints. *Transp Res Part E: Logist Transp Rev* 175:103131
5. Hao J, Lu J, Li X, Tong X, Xiang X, Yuan M, Zhuo HH (2022) Introduction to The Dynamic Pickup and Delivery Problem Benchmark – ICAPS 2021 Competition
6. Zhou Y, Kong L, Yan L, Liu Y, Wang H (2024) A memetic algorithm for a real-world dynamic pickup and delivery problem. *Memetic Computing* 16:1–15

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.